

P2M Project Technical Report

TAI

Secure Multi-tenant Platform For Advanced Scraping

Realised par:

Mohamed Amin AYED – Mohamed Fourat BOUJDARIA

Tutor:

Dr. Maroua NOUIOUA

Table of Contents

1. Introduction

1.1 Context

1.2 Problem statement

1.3 Proposed solution

2. Architecture

2.1 System Architecture

2.1.1 Core Component: API Gateway

2.1.2 Database Architecture

2.2 Security Architecture

2.2.1 Authentication Strategy

2.2.2 Authorization Strategy: Role-Based Access Control (RBAC)

2.2.3 JWT Security and Lifecycle Management

2.2.4 Rate Limiting (3 Layers)

2.2.5 Input Validation

2.2.6 Secrets Management

2.3 Multi-Tenancy Design and Isolation

2.4 Main services

2.4.1 Advanced scraping system

2.4.2 Ingestion service

3. Implementation and Results

3.1 Functional Verification

3.2 Performance Metrics

3.3 Security Penetration Testing Results

4. Results Discussion

5. Learning Outcomes & Future Improvements

6. Conclusion

7. References

1. Introduction

1.1 Context

Nowadays, Data is the new currency, and acquiring it is a crucial part of success in business, no matter the field, but it is especially vital for the procurement industry, because collecting, managing and processing the Procurement Offers efficiently, effectively and securely could be the difference between closing a deal and losing a business opportunity.

For that reason, modern data-driven organizations increasingly rely on automated data collection systems) to extract valuable information from heterogeneous sources such as portals, emails, and APIs. However, these systems face critical security challenges, especially when deployed as multi-tenant platforms.

This project focuses on the design and implementation of a robust multi-tenant scraping platform, with an emphasis on defense-in-depth security, tenant isolation, and high-performance request handling.

1.2 Problem Statement

Scraping platforms often Handle sensitive business data, serve multiple tenants simultaneously and they are exposed to external threats (bots, attackers, malicious inputs).

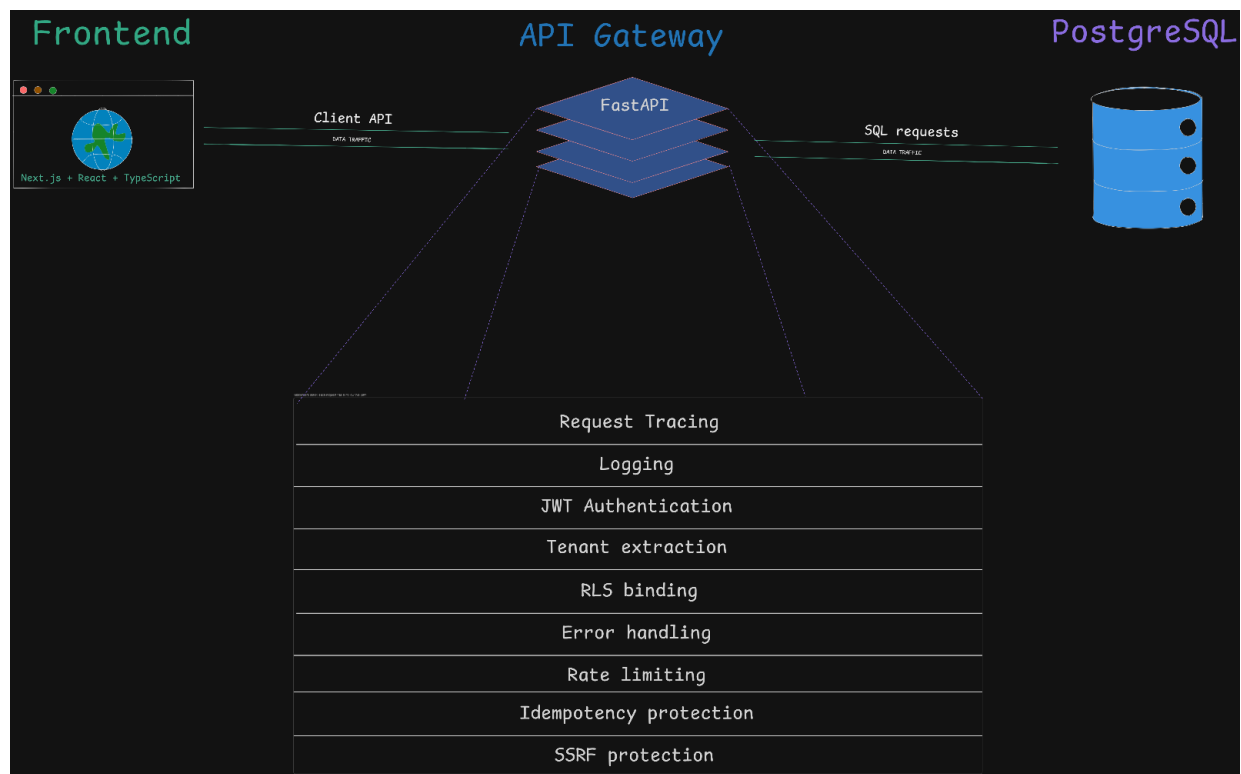
How can we design a **secure, scalable, and high-performance multi-tenant scraping platform** that; Ensures **strict data isolation between tenants**, prevents **web vulnerabilities**, all while maintaining **high throughput and low latency**.

1.3 Proposed Solution: TAI Platform

The proposed solution is TAI: a **secure multi-tenant platform** built in a microservice architecture and around an **API Gateway, Zero-trust authentication, Database-level isolation using PostgreSQL RLS**.

The platform implements strong security mechanisms such as JWT authentication, RBAC, rate limiting, input validation, and Postgres RLS to ensure strict tenant isolation and protect against common web threats and malicious requests. The system is built for scalability and high throughput, and for reliable handling of sensitive business data for multiple tenants.

4. System Architecture



[Figure 1 – Architecture of our Solution]

4.1 Core Component: API Gateway

The API Gateway acts as the **central entry point** for all requests, which is itself a part of our secure by design approach, centralizing the traffic entry allows for tighter control and thus more robust security. This is part of the “secure-by-design” philosophy: centralizing the ingress point to traffic provides tighter control, consistent enforcement, and easier auditing.

4.1.1 Middleware Stack

Each request passes through a structured pipeline:

1. Request tracing (RequestId) :

Assigning a specific id (uuid4) to each incoming and outgoing request for traceability through the X-Request-ID http header.

- Libraries: FastAPI `Request`, Python standard library `uuid.uuid4`, Starlette `BaseHTTPMiddleware`.

2. Logging (ELK-compatible)

Storing each request trace to a logging table to allow debugging and resolving any issues and to ensure non-repudiation. The Logs are stored in a parse-able timestamped format, easy to read and integrate with other tools.

- Libraries: FastAPI `Request`, Starlette `BaseHTTPMiddleware`; Python standard library `logging`, `json`, `time`, `datetime`.

3. JWT Authentication :

Including all required mechanisms to securely authenticate users using the Json Web Token (JWT). It verifies signature, expiration, and builds an identity context (user_id, tenant_id, role) for following-up authorization.

- Libraries: FastAPI `Request`, Starlette `BaseHTTPMiddleware`, Starlette `JSONResponse`; Python standard library `uuid.UUID`.

4. Tenant extraction

Fetching tenant info mainly from JWT claims. Limited fallback to `X-Tenant-ID` for service-to-service calls. Tenant ids are normalized and validated to `uuids`.

- Libraries: FastAPI `Request`, Starlette `BaseHTTPMiddleware`, Starlette `JSONResponse`; Python standard library `uuid.UUID`.

5. RLS binding

Using the extracted tenant context to enforce table-level rules in the database by attaching the tenant context to the DB session to automatically enable PostgreSQL Row-Level Security (RLS) policies.

- Libraries: FastAPI `Request`, Starlette `BaseHTTPMiddleware`;

6. Error handling

Safely and gracefully handling and sending minimal errors to user and verbose enough errors to be logged useful for debugging and keeping failures diagnosable.

- Libraries: FastAPI `HTTPException`, `RequestValidationError`, FastAPI `Request`; Starlette `BaseHTTPMiddleware`, Starlette `JSONResponse`; Python standard library `logging`.

7. Rate limiting

Using the Token-Bucket algorithm while differentiating between authenticated and anonymous users and return (if needed) 429 with rate-limit headers.

- Libraries: FastAPI `Request`, Starlette `BaseHTTPMiddleware`, Starlette `JSONResponse`; Python standard library `dataclasses`, `time`, `math`.

8. Idempotency protection

Safely handle sending the same request multiple time before the backend processes it.

- Libraries: FastAPI `Request`, Starlette `BaseHTTPMiddleware`, Starlette `JSONResponse`; SHA-256.

9. SSRF protection

inspects query/body for URL-like fields and blocks requests that target private/loopback/link-local/reserved IPs, or unsupported schemes.

- Libraries: FastAPI `Request`, Starlette `BaseHTTPMiddleware`, Starlette `JSONResponse`; AnyIO (`anyio.to\_thread.run\_sync`); Python standard library `ipaddress`, `socket`, `urllib.parse`, `json`.

This layered design ensures that **each request is validated, authenticated, isolated, and monitored**.

4.2 Database Architecture: PostgreSQL, SQLAlchemy, and Alembic:

PostgreSQL acts as the database and the mechanism for tenant isolation, because it supports Row Level Security (RLS) directly within the database and does not use application logic for it.

SQLAlchemy and Alembic are used for performing queries, creating sessions, and handling schema migration; however, tenant context is stored safely using PostgreSQL session variables.

5. Security Architecture

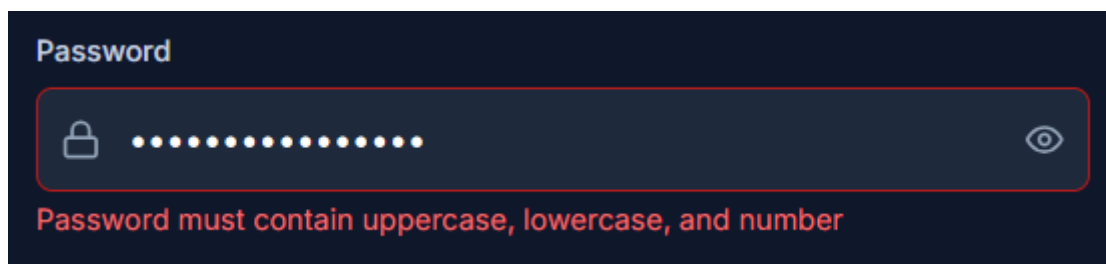
5.1 Authentication Strategy

Each request is considered untrusted from the beginning. The notion of “assumption of authentication based on network location” does not exist; rather, there must be proof of identity in the form of a token with each request.

5.1.1 Passwords:

Each user is required to enter their password to login, which is the first layer of defence, so it must be in a secure format, for which we have the following password policies to enhance the security of the passwords and protect them from attacks like guessing or brute force.

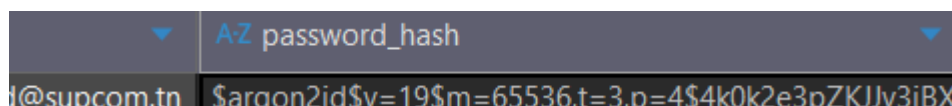
The policies include the obligation to use a password with a length more than eight characters, and to include uppercase and lowercase alphanumerical and symbols. This introduces more complexity for an attacker to access the system without the password.



[Figure 2 – Password policy enforcement]

These passwords are never stored in plain text, each password is hashed using a strong, slow hashing algorithm such as argon2, to make it extremely time and recompute demanding for attackers to brute force the hash to extract the plaintext password.

After hashing, only the hash is stored in the database and used on every credential verification.

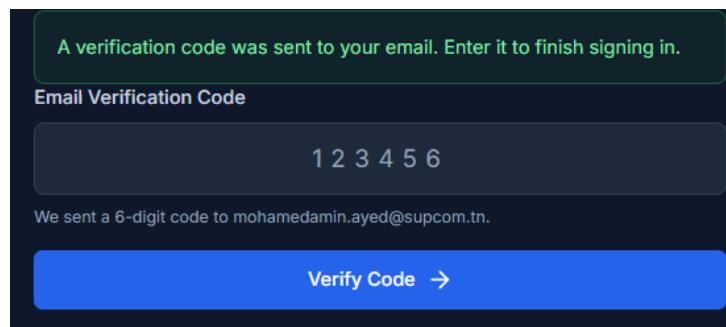


[Figure 2 – passwords stored as hash]

5.1.2 Multifactor authentication:

Nevertheless, the compromise of the user’s credentials is still a possibility. So, we have implemented multifactor authentication to prevent unauthorized access in that case.

Even when the correct email-password combination is entered, the user is required to enter an OTP sent to their email address:



[figure 3 – 2FA with OTP]

5.1.3 JWT for Authentication:

While JWTs are mainly used for authorization, they can be useful in authentication the user with each request as well.

After successfully logging in, a jwt is generated and sent to the user for them to resend it to the server with each request. The JWT includes the user's ID, so the backend can check if the request sender's identity is who they say it is, thus guaranteeing authentication.

5.2 Authorization strategy: RBAC System

The tenant environment has multiple users with different clearances and access privileges, so to address this we implemented a robust Role Based Access Control (RBAC) system.

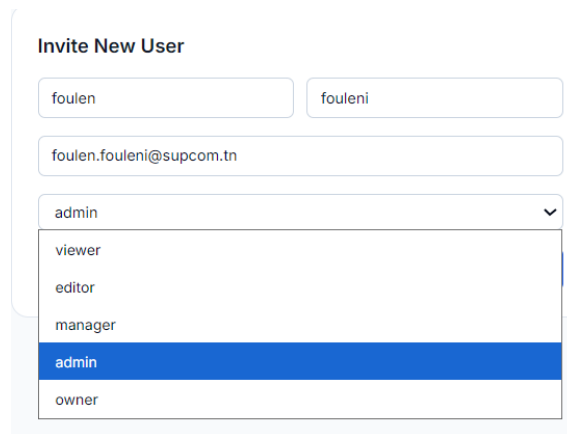
As mentioned earlier, each user has a Json Web Token (JWT) stored client side, containing their claims such as the tenant they belong to, their clearance level and roles.

Each request is only authorized to access data associated with their tenant and accessible under their level of clearance according to their role.

For that we implemented a hierarchical model with 5 roles:

Owner → Admin → Manager → editor → Viewer

- **Owner:** owner acts as a Superadmin user who have full system access, they are the initial tenant creator and owner and they can create or delete tenants and appoint Admins. They are also authorized to make destructive actions such as (tenant deletion, ownership transfer..) for that reason it is advised to have the superadmin account delegated and protected by MFA and extra approvals ultimately used with extra caution.
- **Admin:** Admin users are the tenant-level administrator for day-to-day management (users, settings, API keys, policies, integrations) which is recommended as the account the owner uses to avoid accidental owner-level mistakes.
- **Manager:** Editors can create and modify tenant content (documents, proposals, workflows, configurations) but cannot change security, billing, or ownership settings across tenant.
- **editor:** this is a regular authenticated participant; can perform business actions allowed by tenant policies (submit/modify their own resources, run permitted tasks) and cannot manage other users or tenant-level settings.
- **Viewer:** Read-only access to permitted resources and dashboards without the ability to create, update or delete.



The screenshot shows a web form titled "Invite New User". It contains three input fields: a username field with "foulen", an email field with "foulen.fouleni@supcom.tn", and a role selection dropdown menu. The dropdown menu is open, showing a list of roles: "admin", "viewer", "editor", "manager", "admin" (highlighted in blue), and "owner".

[screenshot 4 - Role management]

Implementing this many roles ensures the **least privilege** concept is enforced, allowing for more control over each user's clearance level.

2FA is required when performing high level security configuration changes for more in-depth security and to be aligned with our zero-trust design.

All security and setting configuration changes are logged for better visibility and accountability.

5.3 JWT Security

Authorization is heavily dependant on JWTs, which are received from users, so this calls for more focus on security, for that reason we have implemented:

- We have implemented multi-algorithm support for our JWTs, so algorithm whitelisting seemed the appropriate as well as choosing secure algorithms (EdDSA, RS256, HS256).
- Refresh token rotation and token blacklisting prevents reuse attacks

5.4 Rate Limiting (3 Layers)

Rate limiting was implemented on different levels with different approaches on each level:

- Network level: Nginx reverse proxy, thanks to its easy to customize configuration, allows for control over the number of requests allowed per IP address.
- Application level: we have implemented the system-wide rate limiting logic per user with a sliding window approach.
- Endpoint-level: this is endpoint specific rate limiting, like sign up requests per email, or otp trials per user...

5.5 Input Validation

Input validation is crucial for safely handling users' input, whether they are authenticated or not, emphasising on our zero-trust approach. And we have achieved that with:

- Strict schemas using the "Pydantic" library which aligns expected input with actual untrusted user input based on pre-configured schemas.
- SQL injection prevention with ORM: Instead of building an SQL query by joining strings together, which is how hackers perform SQL injections, we use Object-Relational Mapping (ORM) which safely plugs user input in SQL queries.

- XSS protection (CSP headers) : using the Content Security Policy (CSP) header limits the risk of Cross Site Scripting (XSS) attacks by whitelisting the sources from which scripts are allowed to be executed.
- File validation: each uploaded file isn't trusted just by its extension, which is checked against a whitelist. The uploaded document goes through multiple file scan tools, like header/magic byte checks, suspicious payloads in the file, filename and size limits.

5.6 Secrets Management

Password hashes and JWT algorithm secrets are stored safely in secret management tools like HashiCorp Vault, and they are subject to automated rotations.

Eliminate reconnaissance by minimizing details in errors

(username exists/doesn't exist → If a user account is associated with the given address, you will receive an email to reset your password)

6. Multi-Tenancy Design

The platform uses **PostgreSQL Row-Level Security (RLS)** as the core isolation mechanism instead of horizontal isolation of databases, which would demand a separate database for each tenant.

This rapidly becomes very expensive and difficult to manage when scaling up to more tenants.

How it works:

1. The user's JWT is sent with each request requiring authorization, the jwt contains tenant_id, which is extracted to populate the user context.
2. DB session sets tenant context to be used as a policy for queries to follow.
3. Postgres then automatically filters queries allowing only those that respect the rls policy.

This ensures **complete isolation even in case of actual compromise** (like an otherwise successful SQL injection).

7. Main services

7.1 Advanced scraping system:

The goal of the platform is helping companies win more procurement offers, so they have to be prompt in getting to the offers first.

That's where our advanced scraping system comes into play; it periodically scraps well known international/national procurement portals for new offers based on the user's preferred sector and other filters.

The advanced aspect of it comes from its ability to deal with anti-scraping websites and dynamic, javascript-heavy webpages.

7.2 Ingestion Service:

This service collects documents from API uploads, Emails, scraping portals..., securely stores data in Minio's object storage and emits events, for the documents to be centralized in the dashboard.

8. Implementation

The development approach follows:

- Thorough research of industry standards and best practices for each new feature.
- Choosing the best approaches based on our business needs and resources.
- Feature-by-feature implementation :
 - Continuous functional and security testing: following request flow with reference to architectural diagrams.

9. Results

9.1 Functional Results

- Tests validate that the **gateway security foundation** and **authentication lifecycle** are fully operational via successful registration operations, efficient Token rotation and management and a systematic tenant isolation process.

9.2 Security Results

Penetration tests confirmed resistance against multiple attacks, and here are some examples:

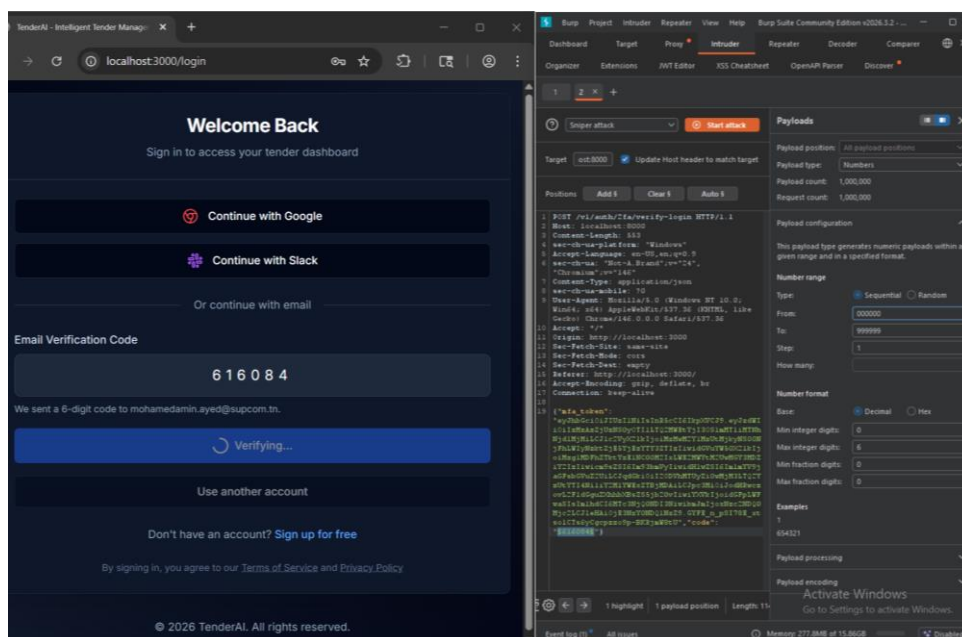
```
nikto -h 172.21.16.1:8080
- Nikto v2.5.0

+ Target IP: 172.21.16.1
+ Target Hostname: 172.21.16.1
+ Target Port: 8080
+ Start Time: 2026-05-25 21:23:53 (GMT1)

+ Server: uvicorn
+ RFC-1918 /: IP address found in the 'x-ratelimit-key' header. The IP is '172.21.25.127'. See: https://portswigger.net/ky/issues/8668388-private-ip-addresses-disclosed
+ /: Uncommon header 'x-request-id' found, with contents: 44437f1a-c91e-4bb8-95a8-d3ea9f8f8dc.
+ /: Uncommon header 'x-ratelimit-algorithm' found, with contents: token_bucket.
+ /: Uncommon header 'x-ratelimit-reset' found, with contents: 59.
+ /: Uncommon header 'x-ratelimit-limit' found, with contents: 10.
+ /: Uncommon header 'x-ratelimit-remaining' found, with contents: 7.
+ /: Uncommon header 'x-ratelimit-key' found, with contents: ip:172.21.25.127.
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ : The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ 5183 requests: 0 error(s) and 0 item(s) reported on remote host
+ End Time: 2026-05-25 21:30:38 (GMT1) (405 seconds)

+ 1 host(s) tested
```

[screenshot 1 - Vulnerability scan with NIKTO]



[screenshot 2 - OTP Bruteforcing with Burpsuite]



- The results show that the platform successfully combines strong security with a scalable multi-tenant architecture. Functional testing confirmed the robustness of the authentication flow, tenant isolation, and middleware security pipeline. PostgreSQL RLS effectively enforced database-level tenant separation, reducing the risk of cross-tenant data leakage. Performance tests also showed good responsiveness despite multiple security layers on each request, including JWT verification, rate limiting, idempotency protection, and SSRF validation. The system maintained an average latency of around 50–120 ms per request with a throughput exceeding 1000 requests per minute per instance, while rate limiting successfully blocked abusive traffic. Security testing also confirmed the effectiveness of the defense-in-depth and zero-trust approach against attacks such as OTP brute-forcing and JWT manipulation attempts.
- The main optimization challenge remains balancing security and performance, since each additional protection layer introduces overhead. Proposed improvements include using Redis for distributed rate limiting and idempotency, reducing unnecessary SSRF checks, and caching JWT verification keys to improve request efficiency.

11. Conclusion

This project demonstrates the feasibility of building a secure, scalable multi-tenant scraping platform with strong isolation guarantees and enterprise-grade security. By combining layered defenses and a robust architecture, the system effectively addresses both security and performance challenges while maintaining strict tenant isolation.

It also shows that security is not a single control but a continuous design decision across all layers - authentication, data access and infrastructure - to be resilient to real world threats without compromising scalability.

13. REFERENCES:

[1] Good Code best practices: <https://realpython.com/python-code-quality>

[2] Pyjwt Documentation: <https://pyjwt.readthedocs.io/en/stable/api.html>

[3] JWT Vulnerabilities: https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/06-Session_Management_Testing/10-Testing_JSON_Web_Tokens
<https://jwtauditor.com/docs/jwt-vulnerabilities-guide.html>
<https://portswigger.net/web-security/jwt>

[4] Advanced web scraping: <https://www.pluralsight.com/resources/blog/guides/advanced-web-scraping-tactics-python-playbook>

[5] Password storage best practices:
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[6] File Upload best practices:
https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

[7] FastAPI Middleware and Performance: <https://fastapi.tiangolo.com/advanced/middleware/>

[8] PostgreSQL Row-Level Security (RLS) for Multi-Tenancy:
<https://aws.amazon.com/blogs/database/multi-tenant-isolation-with-postgresql-row-level-security/>

[9] OWASP's SSRF Prevention Best Practices:
[https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.h
tml](https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html)

[10] Architecting Token-Bucket Rate Limiting: <https://stripe.com/blog/rate-limiters>

[11] Defensive Idempotency Design: [https://microservices.io/patterns/communication-
style/idempotent-consumer.html](https://microservices.io/patterns/communication-style/idempotent-consumer.html)